Advanced Graph Theory

Written by HADIOUCHE Azouaou.

Disclaimer

This document contains the lectures given by Dr. MEHDAOUI.

To separate the contents of the course to actual additions or out of context information, a black band will be added by its side like the one on this comment.

Contents

Chapter 1

Fundamental Concepts

1.1. Definitions & Notations

1.1.1. Types Of Graphs

Definition 1 (Undirected Graph): *An undirected graph G is an ordered pair $G = (V, E)$ where V is a finite set called the vertex set (vertices) and $E \subset \{\{u, v\} \mid u, v \in V\}$ called the set of edges.*

Definition 2 (Directed Graph): *A directed graph G is an ordered pair $G = (V, A)$ where V is the set of vertices and $A \subset V \times V$ called the set of arcs or directed edges.*

Definition 3 (Unweighted Graph): *An unweighted graph G is a graph is a graph where all edges are considered equal.*

Definition 4 (Weighted Graph): *A weighted graph $G = (V, E)$ is a graph where there is a function $w : E \to \mathbb{R}$ where for each edge $e \in E$, the weight of e is $w(e)$.*

1.1.2. Concepts On Vertices & Edges

Definition 5 (Adjacent/Incident/Isolated): *Let $G = (V, E)$ a graph:*

1. *Two vertices $u, v \in V$ are adjacent or neighbors if $\{u, v\} \in E$.*
2. *A vertex v is called incident to an edge e if v is an endpoint of e.*
3. *A vertex with no incident edges is called isolated.*
4. *The order of G is $\# V$.*
5. *The size of G is $\# E$.*
6. *The open neighborhood of $v \in V$ is $\mathcal{N}(v) = \{u \in V \mid \{u, v\} \in E\}$.*
7. *The closed neighborhood of $v \in V$ is $[v] = \mathcal{N}(v) \cup \{v\}$.*
8. *A degree of a vertex v denoted $d(v)$ or $\deg(v)$ is the number of edges incident to v, that is $\deg(v) = \# N(v)$.*
9. *If G is directed then we define the following:*
   1. *The out neighborhood of $v \in V$ is $\mathcal{N}^+(v) = \{u \in V \mid (v, u) \in A\}$.*
   2. *The in neighborhood of $v \in V$ is $\mathcal{N}^-(v) = \{u \in V \mid (u, v) \in A)$.*
   3. *The in/out degree are $\deg^-(v) = \# \mathcal{N}^-(v), \deg^+(v) = \# \mathcal{N}^+(v)$.*
10. *The number of edges incident to v with maximum degree of G, $\Delta(G) = \max_{v \in V} d(v)$ and minimum degree is $\delta(G) = \min_{v \in V} d(v)$.*

Lemma 6 (Handshake): *For an undirected graph $G = (V, E)$ then*

$$\sum_{v \in V} \deg(v) = 2 \# E$$

Proof. Every edge $\{u, v\} \in E$ contributes exactly 1 to $\deg(u)$ and 1 to $\deg(v)$ thus adding 2 to $\sum_{v \in V} \deg(v)$, so all edges counted twice in the sum of all edges. ■

Theorem 7: *The number of vertices with odd degree is even.*

Proof. Let $V_o = \{v \in V \mid d(v) \text{ odd}\}$ and $V_e = \{v \in V \mid d(v) \text{ even}\}$, by the handshake lemma we have

$$2 \# E = \sum_{v \in V} \deg(v) = \sum_{v \in V_o} \deg(v) + \sum_{v \in V_e} \deg(v)$$

given that $2 \# E$ is even and $\sum_{v \in V_e} \deg(V)$ is even then necessarily $\sum_{v \in V_o} \deg(v)$ is even, and since the sum of odd numbers is even if and only if the number of odd numbers is even then $\# V_o$ is even so there is an even number of odd degree vertices. ■

1.1.3. Simple & Multigraphs

Definition 8 (Simple/Multi Graph): *A graph G is said to be simple if it has no loops and no multiple edges. A multigraph is a graph that may have loops and have multiple edges.*

Theorem 9: *In any simple graph G with order $n \geq 2$, there exists at least two vertices with same degree.*

Proof. The possible degrees in a simple graph is $\{0, ..., n-1\}$, however a graph cannot have a vertex with degree 0 and $n-1$ at the same time, so either it has vertices of degree $\{1, ..., n-1\}$ or $\{0, ..., n-2\}$, and since there are n vertices and $n-1$ choices for degrees, then by the pigeonhole principle, there are at least two vertices with the same degrees. ■

Proposition 1.1.3.10: *There are $\binom{n}{2}$ maximum edges in a simple graph.*

Definition 11 (Walk/Trail): *Let $G = (V, E)$ be a graph*

1. *A walk of length k is a sequence $v_1, e_2, v_2, ..., e_k, v_k$ with $e_i = \{v_{i-1}, v_i\} \in E$.*
2. *A trail is a walk with no repeated edges.*
3. *A path is a walk with no repeated vertices.*
4. *A walk is closed if $v_0 = v_k$.*

Theorem 12: *If there is a walk from vertex u to v then there is also a path from u to v.*

Proof. By induction on the length of the walk n. For $n = 1$, it is trivial since any 1-walk is a path, now consider a walk of length n, $v_1, e_1, v_2, ..., v_{n-1}, e_n, v_n$, if $\forall i, j \in [\![1, n]\!], i \neq j \Rightarrow v_i \neq v_j$ then it is a path, otherwise there exists i, j such that $v_i = v_j$, we consider the new walk $v_1, e_1, v_2, ..., e_i, v_i, e_j, ..., e_n, v_n$, this walk has a strictly less length then the original, and thus by induction hypothesis, there is a walk from v_1 to v_n. ■

Definition 13 (Circuit/Cycle):

1. *A circuit is a closed trail.*
2. *A cycle is a closed path.*
3. *A cycle of length k is denoted C_k, unless otherwise specified.*
4. *The length of the cycle is the number of edges in it.*
5. *A graph G is connected if for every pair of vertices $u, v \in V$ there exists a path from u to v, otherwise it is disconnected.*
6. *A subgraph H of G is a graph such that $V(H) \subset V(G)$ and $E(H) \subset E(G)$.*
7. *A connected component of G is a maximum connected subgraph of G.*
8. *A spanning subgraph H of G is a subgraph such that $V(H) = V(G)$.*

Theorem 14: *In a connected graph, for any two longest paths in the graph, there exists a vertex they intersect in.*

Proof. Suppose that there is a graph where two longest paths do not share a vertex, denote them $v_1, e_1, ..., e_{n-1}, v_n$ and $v'_1, e'_1, ..., e'_{n-1}, v'_n$, given that the graph is connected, then there is a path from some v_i to v'_j that does not intersect with the previous paths. Consider the path that goes through the furthest endpoint v_1 or v_n from v_i, the path until v_i, then the path from v_i to v'_j and lastly to the furthest endpoint v'_1 or v'_n from v'_j. ■

Definition 15 (Induced/Complement Graph):

1. *An induced subgraph G' is a subset $S \subset V$ and all edges from G that have both endpoints in S.*
2. *A complement graph $\overline{G}$ of a graph G has the same vertex set V, and $uv \in E\left(\overline{G}\right) \Leftrightarrow uv \notin E(G)$.*

1. $\# E(G) + \# E\left(\overline{G}\right) = \binom{n}{2}$.
2. The number of simple graphs of order n is $2^{\binom{n}{2}}$.
3. The number of spanning subgraphs of K_n with m edges is $\binom{\binom{n}{2}}{m}$.
4. In a graph with n vertices and m edges, the number of induced subgraphs of 2^n and the number of spanning subgraphs is 2^m.

1.2. Graph Representations

There are several ways to represent graphs in computer memory, each with different advantages for various algorithms and applications. We represent it efficiently in memory so that it does the basic operations like edge lookup and neighborhood efficiently and minimize space complexity.

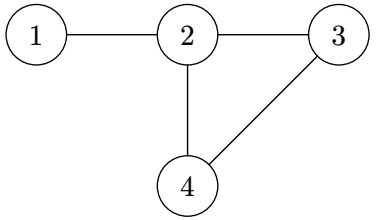
We assume throughout the course that $\#V(G) = n$ vertices and $\#E(G) = m$ edges, vertices are labeled from 1 to n. Graphs may be directed or undirected.

1.2.1. Adjacency Matrix

Let $G = (V, E)$ be a graph with $V = \{v_1, ..., v_n\}$, the adjacency matrix of G is an $n \times n$ matrix $A = (a_{ij})$ where

$$a_{ij} = \begin{cases} 1 \text{ if } v_i v_j \in E(G) \\ 0 \text{ otherwise} \end{cases}$$

Example: Here is a simple graph with its adjacency matrix



$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

For weighted graphs we take the adjacency matrix would be

$$a_{ij} = \begin{cases} w(v_i v_j) \text{ if } v_i v_j \in E \\ \text{None} \quad \text{otherwise} \end{cases}$$

If no edge exists, None, ∞ or 0 can be assigned depending on the application.

1. Space complexity the adjacency matrix is $O(n^2)$.
2. Time to check if an edge exists is $O(1)$.
3. Time to find all neighbors of a vertex is $O(n)$.

1.2.2. Adjacency List

For a graph $G = (V, E)$ the adjacency list representation consists of an array Adj of $\#V$ lists, one for each vertex in V. For each $u \in V$, the adjacency list Adj[u] consists of all the vertices v such that $uv \in E$.

1. Space complexity $O(n + m)$.
2. Time to check if edge exists: $O(d)$ where d is the degree of the vertex.
3. Time to find all neighbors of a vertex: $O(d)$.

For a weighted graph, we just add a couple (v_j, w) if $v_i v_j \in E$ with $w = w(v_i v_j)$.

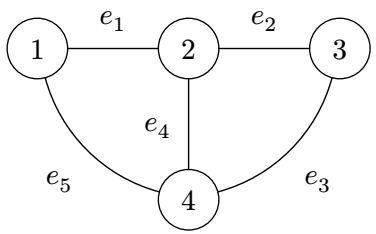
> **Definition 16 (Sparse/Dense Graph):** *Let $G = (V, E)$ a graph, we say that G is a sparse graph if $\#E \ll \#V^2$ and we call it dense if $\#E \approx \Theta(\#V^2)$.*

In sparse graphs the adjacency list is preferred, while in dense graphs the adjacency matrix is preferred.

1.2.3. Incidence Matrix

Given a graph $G = (V, E)$, the incidence matrix M is defined as follows

$$M_{ij} = \begin{cases} 1 \text{ if } v_i \text{ is incident to } e_j \\ 0 \text{ otherwise} \end{cases}$$

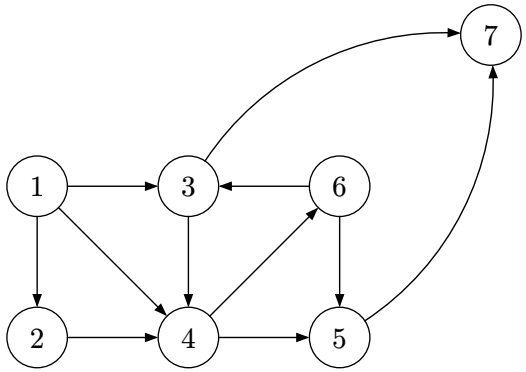


$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}$$

For a directed graph, we define the following representation

$$M_{ij} = \begin{cases} +1 \text{ if } e_j \text{ enters the vertex } v_i \\ -1 \text{ if } e_j \text{ leaves the vertex } v_i \\ 0 \quad \text{otherwise} \end{cases}$$

Example: Consider the following graph



We have the following representations

Adjacency matrix:

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Adjacency list:

$$\begin{aligned} &\text{Adj} = [\\ &\quad 1 \to \{2,3,4\}, 2 \to \{4\}, 3 \to \{4,7\}, \\ &\quad 4 \to \{5,6\}, 5 \to \{7\}, 6 \to \{5\}, 7 \to \{\} \\ &] \end{aligned}$$

Operation	Adjacency Matrix	Adjacency List	Incidence Matrix
Space	$O(n^2)$	$O(n+m)$	$O(n \cdot m)$
Check if edge exists	$O(1)$	$O(d(v))$	$O(m)$
Find all neighbors	$O(n)$	$O(d(v))$	$O(m)$
Add Edge	$O(1)$	$O(1)$	$O(n)$

1.3. Graph Traversal Algorithms

Graph traversal algorithms systematically explores the vertices of the graph. They are fundamental building blocks for many other graph algorithms.

1.3.1. Breadth-First Search

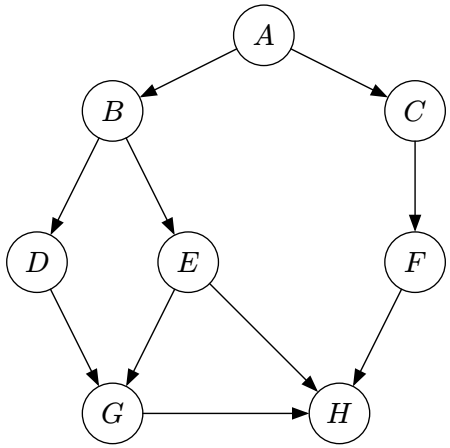
Fundamental graph traversal algorithm explores a graph layer by layer, starting from a source vertex, it visits all its directed neighbors (vertices at distance 1) before moving on to the neighbors of those neighbors, (vertices at distance 2) and so on. BFS uses a Queue data structure (First-In First-Out) to manage the order of exploration. The goal of BFS is to discover all vertices reachable from source s and compute the shortest path distance s to each reachable vertex.

```
1 - procedure BFS(G, s):
2 -    - Create a queue Q.
3 -    - Create a set visited.
4 -    - Create a map distance with values infinity.
5 -
6 -    visited.add(s)
7 -    distance[s] = 0
8 -    Q.enqueue(s)
9 -
10 -   while Q not empty do
11 -     v = Q.dequeue()
12 -     - Process vertex v.
13 -
14 -     for each neighbor w of v
15 -       if w is not visited then
16 -         visited.add(w)
17 -         distance[w] = distance[v] + 1
18 -         Q.enqueue(w)
19 -       end
20 -     end
21 -   end
22 -
23 -   return distance
```

Global complexity with adjacency list is $O(n+m)$, and space complexity is $O(n)$.

Example: Let us apply BFS to the following graph $G = (V, E)$

$$V = \{A, B, C, D, E, F, G, H\}$$
$$E = \{(A, B), (A, C), (B, D), (B, E), (C, F)$$
$$(D, G), (E, G), (E, H), (F, H), (G, H)\}$$



Starting from vertex A we have

1. Initialization:
   - $Q = [A]$
   - $d[B] = \cdots = d[H] = \infty$
   - $P[A] = \cdots = P[G] = \text{None}$
2. Step 1: Dequeue A
   - Neighbors B, C
   - visited $= \{A, B, C\}$
   - $d[B] = d[C] = 1$
   - $P[B] = P[C] = A$
   - $Q = [B, C]$
3. Step 2: Dequeue B
   - Neighbors D, E
   - visited $= \{A, B, C, D, E\}$
   - $d[D] = d[E] = 2$
   - $P[D] = P[E] = B$
   - $Q = [C, D, E]$
4. Step 3: Dequeue C
   - Neighbors F
   - visited $= \{A, B, C, D, E, F\}$
   - $d[F] = 2$
   - $P[F] = C$
   - $Q = [D, E, F]$
5. Step 4: Dequeue D
   - Neighbors G
   - visited $= \{A, B, C, D, E, F, G\}$
   - $d[G] = 3$
   - $P[G] = D$
   - $Q = [E, F, G]$
6. Step 5: Dequeue E
   - Neighbors G, H
   - G already visited
   - visited $= \{A, B, C, D, E, F, G, H\}$
   - $d[H] = 3$
   - $P[H] = E$
   - $Q = [F, G, H]$
7. Step 6: Dequeue F
   - Neighbors H
   - H already visited
   - $Q = [G, H]$
8. Step 7: Dequeue G
   - Neighbors H
   - H already visited
   - $Q = [H]$
9. Step 8: Dequeue H
   - No neighbors
   - $Q = []$

Q is empty, algorithm stops. Which gives us the following results

v	A	B	C	D	E	F	G	H
$d(v)$	0	1	1	2	2	2	3	3
$P[v]$	None	A	A	B	B	C	D	E

Exercise: *Rewrite the BFS algorithm assuming the graph $G = (V, E)$ is represented by its adjacency matrix A and provide its complexity.*

Lemma 17: *Let $G = (V, E)$ be a directed or undirected graph, and let $s \in V$ be a fixed source point. For every edge $(u, v) \in E$, we have*

$$\delta(s, v) \leq \delta(s, u) + 1$$

where $\delta(s, x)$ denotes the length of the shortest path.

Proof. If u is reachable from s, then there exists a shortest path from s to u of length $\delta(s, u)$, appending the edge (u, v) gives a path from s to v of length $\delta(s, u) +$

1. Since $\delta(s,v)$ is defined as the minimum length then necessarily $\delta(s,v) \leq \delta(s,u)+1$. If u is not reachable from s, then trivially $\delta(s,u)+1=\infty$ ■

> **Lemma 18:** *Suppose that during the execution of BFS on a graph $G=(V,E)$, the queue Q contains the vertices $[v_1,...,v_r]$ where v_1 is the head of Q and v_r is the tail of Q. Then the following holds:*
> 1. $d(v_r) \leq d(v_1)+1$.
> 2. $\forall i \in [\![1,...,r-1]\!], d(v_i) \leq d(v_{i+1})$.

Proof. By induction on the number of queue operations.

- Base case: initially, the queue contains the source vertex s, the lemma is trivial.
- Assume the lemma holds before queue operations, we must show that it continues to hold after a dequeue.
  - Dequeue operation: if the head v_1 is dequeued then v_2 becomes the new head (unless the queue becomes empty, in which case the statement remains true), by induction hypothesis we have that $d(v_1) \leq d(v_2)$ since $d(v_r) \leq d(v_1)+1$ we obtain that $d(v_r) \leq d(v_2)+1$ and all the other inequalities remain.
  - Enqueue operation: suppose a vertex v is discovered while scanning the adjacency list of some vertex, hence $d(v)=d(u)+1$ and enqueue v at the tail of the queue. At the moment, u has already been dequeued. By the induction hypothesis the current head v_1 satisfies $d(u) \leq d(v_1)$. Therefore, $d(v)=d(u)+1 \leq d(v_1)+1$. Furthermore, the previous tail $d(v_r) \leq d(u)+1=d(v)$. Therefore, the lemma holds for all queue operations.

Which proves this lemma. ■

> **Theorem 19:** *Let $G=(V,E)$ be an unweighted graph for any vertex $v \in V$ reachable from the source s, the distance $d[v]$ assigned by the BFS algorithm is equal to the shortest path distance $\delta(s,v)$.*

Proof. Using induction on the shortest path $\delta(s,v)=k$, because for $k=0$ the only vertex v such that $\delta(s,v)=0$ is s itself. BFS initialize $d[s]=0$ thus $d[s]=\delta(s,s)=0$. By induction, assume that forall vertices u with $\delta(s,u)=k$, BFS assign correctly $d[u]=k$, consider a vertex v such that $\delta(s,v)=k+1$. By definition of shortest distance there exists a vertex u such that $(u,v) \in E$ and $\delta(s,u)=k$. By inductive step $d[u]=k$. When u is dequeued, the algorithm examines all neighbors of v is undiscovered. BFS sets $d[v]=d[u]+1$. Since BFS explores vertices in non-decreasing order of distance, v cannot have been discovered at a distance smaller than $k+1$ (otherwise a shortest path to v would be contradicting $\delta(s,v)=k+1$. Thus, the first time v is reached, it is assigned $d[v]=k+1$. ■

> **Exercise:** *The diameter of a tree $T=(V,E)$ is defined as*
>
> $$\max_{u,v \in V} \delta(u,v)$$
>
> *that is the longest of all shortest path distances in the tree. Give an efficient algorithm to compute the diameter of a tree and analyze its complexity.*

```
 1 - procedure BFSmodified(T, s):
 2 -   - create a queue Q
 3 -   - create distances d
 4 -   - initialize d for all vertices to infinity
 5 -
 6 -   d[s] = 0
 7 -   Q.enqueue(s)
 8 -   while Q not empty do
 9 -     v = Q.dequeue()
10 -     for each neighbor w of v do
11 -       if d[w] = infinity then
12 -         d[w] = d[v] + 1
13 -         Q.enqueue(w)
14 -       end
15 -     end
16 -   end
17 -
18 -   x = argmax(d)
19 -   return (x, d[x])
```

Now that we have the modified BFS, we can use it to search the diameter of the tree, that is, we start with a random vertex, go to the furthest vertex, and from that vertex, we apply the same thing again, which will give us the longest path.

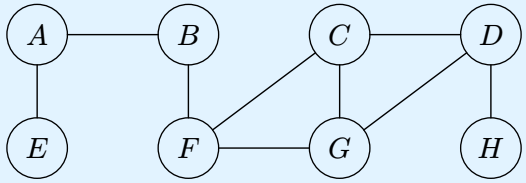
```
1 - procedure TREEdiameter(T)
2 -    - chose an arbitrary vertex s in V
3 -    (u, _) = BFSmodified(T, s)
4 -    (v, d) = BFSmodified(T, u)
5 -    return d
```

The complexity of the pseudocode $O(n)$.

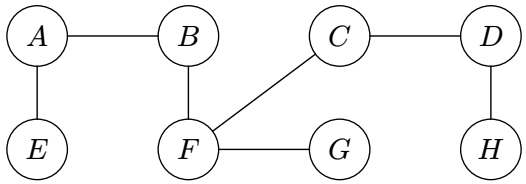
The BFS builds a BFS-Tree as it searches the graph. The tree corresponds to the p attributes. For a graph $G = (V, E)$ with sources. Let $G_p = (V_p, E_p)$ where $V_p = \{v \in V \mid p(v) \neq \text{Nil}\} \cup \{s\}$ and $E_p = \{(p(v), v) \mid v \in V_p \setminus \{s\}\}$. The predecessor subgraph G_q is a BFS tree if V_p consists of the vertices reachable from s and for all $v \in V_p$, the subgraph G_p contains a unique simple path from s to v, that is the shortest path from s to v in G. A BFS-Tree is in fact a tree, since it is connected by construction and $|E_p| = |V_p| - 1$.

Exercise: *Show that if a graph is connected and it has $n - 1$ edges then the graph is a tree.*

Exercise: *Run the BFS algorithm on the following graph and give the BFS tree starting from vertex A.*



Going through the same procedure as the previous example to get



Exercise: *The square of a directed graph $G = (V, E)$ is the graph $G^2 = (V, E^2)$ with E^2 defined as, $uv \in E^2 \Leftrightarrow uv \in E$ or $\exists w \in V, uw, wv \in E$. Write an efficient algorithm for computing G^2 from G for both adjacency list and adjacency matrix representations of G. Analyze the complexity of the algorithm.*

Exercise: *The transpose of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$ where $E^T = \{uv \mid u, v \in V, vu \in E\}$. Write an efficient algorithm for computing G^T for both representations. Analyze its complexity.*

1.3.2. Depth-First Search

DFS is a graph traversal algorithm that explores as far as possible along each branch before backtracking. DFS uses stack data structure (LIFO) to manage the exploration path.

There are two ways to implement DFS, either recursive or iterative.

- **Recursive**

```
 1 - procedure DFSvisit(G, v, visited):
 2 -   visited.add(v)
 3 -
 4 -   for each neighbor w of v do
 5 -     if w not visited then
 6 -       P[w] = v
 7 -       DFSvisit(G, w, visited)
 8 -     end
 9 -   end
10 -
11 - procedure DFS(G):
12 -   - create a set visited
13 -   - create a parent array P
14 -
15 -   for each v in G do
16 -     if v not visited then
```

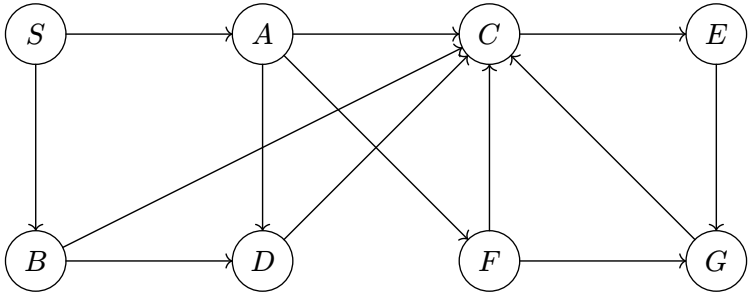
```
      DFSvisit(G, v, visited)
    end
  end
```

- **Iterative**

```
procedure DFS(G, s):
  - create a stack S
  - create a set visited
  - create a parent array P

  while S not empty do
    v = S.pop()
    if v not visited then
      visited.add(v)
      - Process vertex v
      for neighborhood w of v in reverse order do
        if w not visited then
          P[w] = v
          S.push(w)
        end
      end
    end
  end
```

Example: We consider the following graph, and we apply the DFS starting from the vertex S.



1. Initialization:
   - $Q = [S]$
   - visited $= \{\}$
   - $P[S] =$ None
2. Step 1: Pop S
   - Neighbors A, B
   - visited $= \{S\}$.
   - $P[A] = S, P[B] = S$.
   - $S = [B, A]$.
3. Step 2: Pop A
   - Neighbors D, F.
   - visited $= \{S, A\}$.
   - $P[D] = A, P[F] = A$.
   - $S = [B, F, D]$.
4. Step 3: Pop D
   - Neighbors C.
   - visited $= \{S, A, D\}$.
   - $P[C] = D$.
   - $S = [B, F, C]$.
5. Step 4: Pop C
   - Neighbors E.
   - visited $= \{S, A, D, C\}$.
   - $P[E] = C$.
   - $S = [B, F, E]$.
1. Step 5: Pop E
   - Neighbors G.
   - visited $= \{S, A, D, C, E\}$.
   - $P[G] = E$.
   - $S = [B, F, G]$.
2. Step 6: Pop G
   - Neighbors C, F.
   - All visited.
   - $S = [B, F]$.
3. Step 7: Pop F
   - Neighbors C, G.
   - All visited.
   - visited $= \{S, A, D, C, E, G, F\}$.
   - $S = [B]$.
4. Step 8: Pop B
   - Neighbors C, D
   - All visited.
   - $S = []$.

Discovering And Finishing Time:

Definition 20 (Discovering/Finishing Time): *We define the following functions:*
- $d[U]$*: the time when* U *was visited first.*
- $f[U]$*: the time when the exploration of* U *adjacency list is completed.*

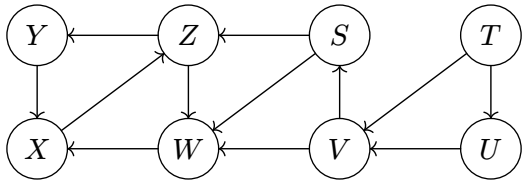
It is clear that $d[U] < f[U]$ for any vertex U.

Definition 21 (Edge Classification): *Let (u, v) be an edge of a direct graph explored during DFS, then we define the following classes:*

- *Tree edge: if v is first discovered when exploring u, $P[v] = u$.*
- *Back edge: if v is an ancestor of u.*
- *Forward edge: if v is a descendent of u, $d[u] < d[v]$.*
- *Cross edge: $d[u] > d[v]$ and u, v are in different branches.*

Pranthesis Structure: A fundamental property of DFS is that discovery and finishing times of vertices follow a parenthesis structure, more precisely, if we denote the discovering of a vertex by (u and its finishing time by u) then the sequence of events produced by a DFS forms a properly parenthesized expression.

Example:



we get the following expression

$$(S(Z(Y(XX)Y)(WW)Z)S)(T(U(VV)U)T)$$

Theorem 22: *Int a DFS of a graph G for any two vertices u and v exactly one of the following holds:*

1. *$[d[u], f[u]$ and $[d[v], f[v]]$ are entirely disjoint.*
2. *$[d[u], f[u]] \subset [d[v], f[v]]$.*
3. *$[d[v], f[v]] \subset [d[u], f[v]]$.*

Proof. Without loss of generality, assume $d[u] < d[v]$, since all timestamps are distinct (the clock increments before each assignment, so all $2|V|$ timestamps are distinct). There are two subcases according to wether $d[v] < f[u]$ or not.

- Case 1: $d[v] < f[u]$, so v is discovered while u is still open (not finished), which implies that v is a descendent of u. Moreover, since v is discovered after u it is finished before u, hence $[d[v], f[v]] \subset [d[u], f[u]]$.
- Case 2: $d[v] > f[u]$ then $d[u] < f[u] < d[v] < f[v]$, thus the intervals are entirely disjoint no vertex is a descendent of another, it is obtained by the symmetry $d[v] < d[u]$. ■

Theorem 23 (Cycle Detection): *A directed graph contains a cycle if and only if DFS discovered a back edge.*

Proof. $\Leftarrow$ Suppose DFS discovers a back edge uv, by definition of back edge in DFS, the vertex v is an ancestor of u in the DFS tree. Therefore, there exists a path from v to u consisting of tree edges in DFS tree, together with the edge uv, we get a cycle. $\Rightarrow$ suppose the graph contains a directed cycle $v_1 \to v_2 \to ...v_k \to v_1$, let v_1 be the first vertex of this cycle discovered by DFS, when DFS visits v_1 all other vertices of the cycle are still undiscovered, DFS will explore along the cycle through tree edge until it reaches the vertex v_K that has an edge back to v_1, since v_1 is still active in the recursive stack, this edge points to an ancestor in DFS tree therefore it is a back edge. ■

Exercise 24: *Prove the following theorem.*

> **Theorem:** *In a DFS of an undirected graph G, every edge of G is either a tree edge or a back edge.*

1.3.3. Application Of Traversal Algorithms

- **Finding The Shortest Path:** using BFS we can find the shortest path from a starting vertex s to an end vertex e.

```
1 - procedure BFSshortpath(G, s, e):
2 -   BFS(G, s)
3 -   if P[e] = NIL and e ≠ s then return "No Path" end
4 -   return BFSreconstructpath(P, s, e)
5 -
6 - procedure BFSreconstructpath(P, s, e):
7 -   path = []
8 -   current = e
9 -
```

```
  while current ≠ NIL do
    path.add(current)
    current = P[current]
  end

  return path
```

- **Connected Components:** a connected components of an undirected graph G is a maximal subset $C \subset V$ such that for every pair $u, v \in C$, there is a path from u to v in G. We can use BFS/DFS to find connected components.

```
procedure FSfindcomponents(G):
  - initialize components array to -1 for vertices.
  count = 0

  for vertex v in G do
    if component[v] = -1 then
      count = count + 1
      FSlabelcomponent(G, v, count)
    end
  end

  return count, components

procedure FSlabelcomponent(G, s, label):
  - use BFS/DFS starting from s
  for vertex v in visited do
    components[v] = label
  end
```

- **Topological Sort:** Let G be a direct acyclic graph "DAG" is a linear ordering of all its vertices such that $uv \in E(G) \Rightarrow u \leq v$.

Lemma 25: *A directed graph G is acyclic if and only if DFS produces no back edge.*

Proof. $\Rightarrow$ if DFS produces a back edge uv then there is a cycle. $\Leftarrow$ if G contains a cycle, DFS will eventually discover a back edge. ■

```
procedure DFStopsort(G):
  - initialize empty list L
  - Run DFS in G and compute the finishing times f[v]
  - as each vertex if finished insert it in front of L
  return L
```

- **Trees:**

Definition 26 (Tree): *A tree is a connected non-cyclic graph.*
- *we say it is rooted if it has one designated vertex called a root.*
- *the depth of a vertex is the length from the root to the vertex.*
- *the height of the tree is the maximum depth of any vertex.*
- *spanning tree: a spanning tree of a connected graph G is a subgraph T such that T is a tree and contains all vertices of G.*

Theorem 27: *Every connected graph has at least one spanning tree.*

Proof. Let G a connected graph, for any cycle, remove an edge from the cycle, the graph remains connected. Repeat until no cycles remain, which is a tree. ■

```
procedure FSspanningtree(G, s):
  - run BFS/DFS on G starting from S
  return FSconstructtree(P)

procedure FSconstructtree(P):
  - Initialize T = (V, {})
  for v in V do
    if P[v] ≠ NIL then
      - add edge(P[v], v) to T
    end
  end

  return T
```

Exercise 28 (The Word Transformer):

1. *Each word $D \cup \{S\}$ is a vertex. An edge links two vertices (words) differencing in exactly one letter.*
2. *All edges are weight 1, any valid transformation sequence is a path in G and the shortest path gives the minimum chain length which BFS guarantee.*
3.

1.4. Minimum Spanning Tree

Given a connected weighted graph $G = (V, E, W)$ where $W : E \to \mathbb{R}$. A minimum spanning tree is a spanning tree T such that $\sum_{e \in E(T)} W(e)$ is minimized.

1.4.1. Prim's Algorithm

Prim's algorithm grows the MST one vertex at a time from a source s, at every step it selects the cheapest edge.

```
1 - procedure PrimMST(G, r):
2 -   for each v in V do
3 -     key[v] = ∞
4 -     P[v] = NIL
5 -   end
6 -
7 -   key[r] = 0
8 -   Q = V
9 -
10 -  while Q not empty do
11 -    u = extractmin(Q)
12 -    for each neighbor v of u do
13 -      if v in Q and w(uv) < key(v) then
14 -        key[v] = w(uv)
15 -        P[v] = u
16 -      end
17 -    end
18 -  end
19 -
20 -  return P
```

1.4.2. Kruskal's Algorithm

Kruskals algorithm process the edges in increasing weight order and greedily includes each edges in the MST as long as it does not form a cycle with edges already selected.

```
1 - T = ∅
2 - for v in V do makeset(v) end
3 -
4 - - sort E in non-decreasing order of weight
5 -
6 - for edge (u, v) in E do
7 -   if findset(u) ≠ findset(v) then
8 -     T = T ∪ {(u, v)}
9 -     union(u, v)
10 -  endif
11 - end
```

1.5. Binary Search Tree

Definition 29 (Binary Tree): *Binary tree node x stores a key x.key and three attributes, x.left, x.right (children) and x.parent, if any of the values is NIL indicates their absence.*

The unique node with x.parent $=$ NIL is the root. A node with x.left $= x$.right $=$ NIL is a leaf. The height of the tree is the length of the longest root to leaf path.

Definition 30 (BST Property): *A binary tree is said to satisfy the BST property if for every node x, then for all y in the left subtree of x, y.key $\leq$ x.key and for all z in the right subtree of x, z.key $\geq x$.key.*

- The in-order traversal: $L \to x \to R$.
- The pre-order traversal: $x \to L \to R$.
- The post-order traversal: $L \to R \to x$.

1.5.1. BST Operations

- **Search:**

```
1 - procedure BSTsearch(x, k):
2 -   if k < x.key then return BSTsearch(x.left, k) end
3 -   if k > x.key then return BSTsearch(x.right, k) end
4 -   return x
```

- **Minimum/Maximum:**

```
1 - procedure BSTminimum(x):
2 -   while x.left != NIL do x <- x.left end
3 -   return x
```

- **Predecessor/Successor:**

```
1 -   procedure BSTpredecessor(x):
2 -     if x.left != NIL then
3 -       return BSTmaximum(x.left)
4 -     end
5 -
6 -     y = x.parent
7 -     while y != NIL and x = y.left do
8 -       x = y
9 -       y = y.parent
10 -    end
11 -    return y
```

Exercise:

- *Write the pseudocode for the following operations:*
  1. *Find a successor of x.*
  2. *Insert a key into the BST.*
  3. *Delete a key from the BST.*
- *Calculate the complexity of search, insert, delete, minimum, maximum each run in $O(h)$ time where h is the height of the tree.*

1.6. Graph Coloring

Definition 31 (Proper Vertex Coloring/Chromatic Number): *A proper vertex coloring of a graph G is a function $c : V(G) \to \{1, ..., k\}$ such that $\forall (u, v) \in E(G), c(u) \neq c(v)$. The minimum number of colors for which we can obtain a proper vertex coloring exists, denoted $\chi(G)$.*

1.6.1. Vertex Coloring

Theorem 32:

- $\chi(G) \geq \omega(G)$ *where* $\omega(G)$ *is the clique number.*
- $\chi(G) \leq \Delta(G) + 1$.
- $\chi(G) \leq n$.

```
1 - procedure GreedyColoring(G):
2 -   - initialize all vertices to uncolored
3 -   for v in V(G) do
4 -     - find the smallest color not used by any neighbor
5 -     - assign this color for v
6 -   end
```

1.6.2. Edge Coloring

Edge coloring is an assignment of colors to the edges of a graph such that no adjacent edges have the same color. The chromatic $\chi'(G)$ of a graph G is the minimum number of colors needed for a proper edge coloring.

```
1 - procedure EdgeColoring(G):
2 -   - initialize the color of each edge to uncolored
3 - for each edge e1 in E do
4 -   create an empty set called used_color
5 -   for each edge e2 adjacent of e1 do
6 -     if e2 is already colored then
7 -       - add the color e2 to used_color
8 -     end
9 -   end
```

```
 10 -    - choose the smallest positive color that is not used
in used_colors
 11 -    - assign this color to edge e1
 12 - end
```

Chapter 2

Advanced Problems In Graph Theory

In many real world scenarios, we're interested not just in whether a path exists between points, but finding the most efficient or optimal path. For example, the shortest path problem: given a weighted graph, find a path between two vertices such that the sum of the weights of its edges is minimized.

2.1. Djikstra's Algorithm

Djikstra's algorithm solves the single source shortest path problem. The algorithm works for weighted graphs with non-negative weighted edges.

```
procedure Djikstra(G, s):
  - initialize distance d[v] = ∞ for all vertices v
  - initialize d[s] = 0.4
  - create a parent array P
  - create a distance priority queue with all vertices

  while Q not empty do
    - extract vertex u with minimum distance from Q
    for each neighbor v of u do
      alt = d[u] + weight(u, v)
      if alt < d[v] then
        d[v] = alt
        P[v] = u
        - update priority v in Q
      end
    end
  end
```

The complexity of the Djikstra algorithm is dependent on the implementation of the extract min function

- if it is array-based with normal search, the extraction is $O(n)$ and thus the total complexity is $O(n^2 + m) \simeq O(n^2)$.
- if it is implemented as binary heap, then extramin will be in $O(\log(n))$ and decreasing key is $O(\log(n))$ and thus the total complexity is $O((n + m)\log(n))$.

> **Theorem 1 (Djikstra's Algorithm Correctness):** *Djikstra's algorithm gives the shortest algorithm on a postive weighted graph.*

Proof. We proceed by induction on S.

- $S = \emptyset$, the first extracted vertex is S, $d[s] = 0 = \delta(s, s)$, holds.
- Assume for all vertices $x \in S$ we have $d[x] = \delta(s, x)$. Let $v \in S$ be the next vertex extracted from the priority queue. By definition $d[v] = \min_{x \notin S} d[x]$, assume by contradiction that $d[v] > \delta(s, v)$, consider a shortest path from s to v and $(x, y))$ be the first edge on this path such that $x \in S$, $y \notin S$, by inductive hypothesis $d[x] = \delta(s, x)$, when x is processed, the relaxation step ensures that $d[y] \leq d[x] + w(x, y) = \delta(s, x) + w(x, y) = \delta(s, y)$. Since all weights are non-negative, the remaining path from y to v cannot decrease the distance so $\delta(s, y) \leq \delta(s, v)$ thus $d[y] \leq \delta(s, v)$. But since v is chosen to be the minimum then $d[x] \leq d[y]$ since $y \notin S$, thus $d[v] \leq d[y] \leq \delta(s, y) \leq \delta(s, v)$ which contradicts the assumption of $d[v] > \delta(s, v)$. ■

2.2. Bellman-Ford Algorithm

Which Djikstra's algorithm works efficiently for graphs with non-negative weights if fails when negative weights are present. The Bellman-Ford algorithm solves this single source.

```
procedure Bellman-Ford(G, s):
  - initilize distances d[v] to infinity
  for v vertex in G do
    d[s] = 0
  end

  for i = 1 to #V - 1 do
```

```
    for each edge (v, u) with weight w in G do
      if d[v] + w < d[v] do
        d[v] = d[u] + w
        p[v] = u
      end
    end
  end

  for each edge (u, v) with weight w in G do
    if d[u] + w < d[v] then
      return "negative cycle"
    end
  end

  return d[], p[]
```

Exercise 2: *Prove the correctness of Bellman-Ford algorithm (assuming that there are no negative cycles).*

Let p be the shortest path from s to v

- base case $d[s] = 0 = \delta(s, s)$.
- Assume that all shortest path of length k are determined in iteration 1, all shortest path of length 1 are determined, same for the next iteration, then continuing each iteration, we have that $d[v] \leq d[v_k] + w(v_k, v)$. Hence all shortest path of length $\leq k + 1$ edges are correctly identified.

2.3. Floyd-Wrashall Algorithm

Sometimes we need to find shortest paths between all pairs of vertices while we could run Djikstra's algorithm $\# V$ times, the Floyd-Warshall algorithm offers an elegant solution.

```
procedure FloydWarshall(G):
  - Create distance matrix
  - d[i, j] = w(i, j), d[i, i] = 0
  for k = 1 to #V do
    for j = 1 to #V do
      for i = 1 to #V do
        if d[i, k] + d[k, j] < d[i, j] then
          d[i, j] = d[i, k] + d[k, j]
        end
      end
    end
  end

  return d
```

Exercise 3: *Add an instruction to the pseudocode to reconstruct all shortest paths.*

2.4. Maximum Flow & Min Cuts

Many engineering and optimization problems can be modeled as movement of a material through a network of conducts from a source to a sink. The fundamental algorithmic question is what is the maximum rate at which material can be shipped from source to sink, respecting the capacity of each conduct.

Definition 4 (Flow Network): *A flow network is a directed graph $G = (V, E)$ together with a capacity function $C : V \times V \to \mathbb{R}$ satisfying:*

1. $\forall uv \in E, C(u, v) \geq 0$.
2. $\forall uv \notin E, C(u, v) = 0$
3. *No self loops, no anti-parallel pairs.*
4. *Two distinguied vertices a source s and sink t.*
5. *Every vertex $v \in V$ lies on a source path $s \to v \to t$.*

Definition 5 (Flow): *Let $G = (V, E)$ be a directed graph and $C : V \times V \to \mathbb{R}$ a capacity function. A flow in G is a function $f : V \times V \to \mathbb{R}$ satisfying the two following properties*

- *Capacity constraint:* $\forall u, v \in V, 0 \leq f(u, v) \leq C(u, v)$.
- *Flow conservation:* $\forall u \in V \setminus \{s, t\}, \sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u)$

The value of a flow is $|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$.

What if the problem has several sources $\{s_1, ..., s_n\}$ and several sinks $\{t_1, ..., t_n\}$? Given a flow f in G, the residual network $G_f = (V, E_f)$ captures how additional flow can be pushed away each edge in either directions.

Definition 6 (Residual Capacity/Residual Network): *Let $G = (V, E)$ be a flow network, for vertices $u, v \in V$ the residual capacities are defined as*

$$C_f(u,v) = \begin{cases} C(u,v) - f(u,v) & \text{if } uv \in E_f \text{ (Forward residual edge)} \\ f(v,u) & \text{if } vu \in E_f \text{ (Backward residual edge)} \\ 0 & \text{otherwise} \end{cases}$$

The residual network is $G_f = (V, E_f)$ where

$$E_f = \{(u,v) \in V \times V \mid C_f(u,v) > 0\}$$

Definition 7 (Augmenting Path): *Let G be a flow network, an augmenting path p is a simple path from s to t in the residual network G_f, the residual capacity of p is $C_f(p) = \min_{(u,v)\in p} C_f(u,v)$.*